Relatório Técnico de Entrega 7 — API REST em Node.js, Consultas ao Redis e Tratamento de Erros HTTP

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Novembro / 2026

1. Resumo Executivo da Entrega

A **Entrega 7** implementa e documenta a camada de **API HTTP/JSON REST em Node.js com TypeScript (Express)** responsável por disponibilizar os dados da solução IoT ao sistema web (Dashboard SPA em React).

A arquitetura da API foi projetada com integração direta ao banco de dados chave-valor **Redis** (`kvStore`), fornecendo endpoints dedicados para consulta do **estado atual** do dispositivo, do **histórico de telemetria** e das informações de **presença / último contato (`lastSeen`)**, além de proteção por **Autenticação Básica (HTTP Basic Auth)** e um middleware centralizado de **tratamento de erros e códigos de status HTTP** (`200`, `400`, `401`, `404`, `503`, `504` e `500`).

2. Tabela de Conformidade dos Produtos Esperados

#	Produto Esperado	Requisito da Disciplina	Status	Implementação / Evidência
1	**API desenvolvida em Node.js**	Servidor Express + TypeScript modularizado	**100% Atendido**	`backend/src/index.ts` e `backend/src/api/routes.ts`
2	**Endpoint para consulta do estado atual**	Retorna a última leitura e estado do atuador	**100% Atendido**	`GET /api/devices/:id` e `GET /api/devices/:id/state`
3	**Endpoint para consulta do histórico**	Retorna série temporal com limite e filtro por data	**100% Atendido**	`GET /api/devices/:id/history` (`routes.ts:74-88`)
4	**Endpoint de presença ou último contato**	Retorna `status` (`online`/`offline`), `lastSeen` e IP	**100% Atendido**	`GET /api/devices/:id/presence` (`routes.ts:56-72`)
5	**Respostas estruturadas em JSON**	Padronização de contratos JSON com tipagem estrita	**100% Atendido**	Serialização JSON em todas as rotas de sucesso e erro
6	**Tratamento de erros e códigos HTTP**	Códigos `400`, `401`, `404`, `503`, `504` e `500`	**100% Atendido**	`backend/src/middleware/error_handler.ts` e `routes.ts`
7	**Integração entre API e Redis**	Leitura assíncrona de `SET`, `STRING` e `LIST` no Redis	**100% Atendido**	`backend/src/storage/kv_store.ts` (`getDeviceRecord`, `getDeviceHistory`)
8	**Evidência das consultas funcionando**	Testes automatizados Vitest e consultas cURL reais	**100% Atendido**	Seção 6 deste relatório e `backend/tests/e2e.test.ts`

3. Arquitetura da Camada de API e Integração com o Redis

■ Log de Execução / Diagnóstico

```
[Sistema Web React / Cliente HTTP REST]
       ■
       ■  Requisições HTTP + Basic Auth (Authorization: Basic ...)
       ▼
[Servidor Express Node.js (Porta 5001) — /api]
       ■■■■ GET  /api/health               ■■■ Diagnóstico do Gateway,
           MQTT e Redis (200 OK)
       ■■■■ GET  /api/devices              ■■■ Lista todos os
           dispositivos via SMEMBERS "devices:index"
       ■■■■ GET  /api/devices/:id          ■■■ Registro completo
           (Presença + Estado Atual)
       ■■■■ GET  /api/devices/:id/state    ■■■ Estado atual de
           telemetria e atuador (200 / 404)
       ■■■■ GET  /api/devices/:id/presence ■■■ Status online/offline,
           lastSeen e IP (200 / 404)
       ■■■■ GET  /api/devices/:id/history  ■■■ Histórico temporal via
           LRANGE "device:{id}:history"
       ■■■■ POST /api/devices/:id/command  ■■■ Envio de comando MQTT
           com espera de confirmação ACK
       ■
       ▼
[Camada de Persistência (kvStore — ioredis)]
       ■
       ▼
[Banco Chave-Valor Redis (iot-redis:6379)]
  • SET    "devices:index"            ■■■ Índice de dispositivos
    registrados
  • STRING "device:{id}:presence"     ■■■ JSON com status, lastSeen, ip
    e state atual
  • LIST   "device:{id}:history"      ■■■ Lista FIFO com até 5.000
    amostras históricas
```

4. Especificação dos Endpoints e Códigos de Status HTTP

4.1 Tabela de Endpoints da API REST

Método	Endpoint	Descrição	Chave Redis Consultada	Códigos HTTP
`GET`	`/api/health`	Status operacional da API, conexão MQTT e Redis	—	`200`
`GET`	`/api/devices`	Lista todos os dispositivos e seus estados	`devices:index` + `device:{id}:presence`	`200`, `401`
`GET`	`/api/devices/:id`	Consulta o registro completo e estado atual	`device:{id}:presence`	`200`, `401`, `404`
`GET`	`/api/devices/:id/state`	Consulta especificamente o estado atual dos sensores	`device:{id}:presence` (`.state`)	`200`, `401`, `404`
`GET`	`/api/devices/:id/presence`	Consulta presença (`online`/`offline`) e `lastSeen`	`device:{id}:presence`	`200`, `401`, `404`
`GET`	`/api/devices/:id/history`	Consulta o histórico de telemetria (`?limit=&date=`)	`device:{id}:history`	`200`, `401`, `404`
`GET`	`/api/devices/:id/history/export`	Exporta o histórico de telemetria em formato CSV	`device:{id}:history`	`200`, `401`
`POST`	`/api/devices/:id/command`	Aciona o atuador remoto aguardando ACK do ESP32	`device:{id}:presence`	`200`, `400`, `401`, `503`, `504`

4.2 Mapeamento de Tratamento de Erros e Códigos HTTP

Código HTTP	Identificador (`error`)	Cenário Tratado pela API
200 OK	—	Consulta ou comando processado com sucesso, retornando payload JSON estruturado.
400 Bad Request	`VALIDATION_ERROR`	Parâmetros ou corpo JSON inválidos rejeitados pelo schema Zod no `errorHandler`.
401 Unauthorized	`UNAUTHORIZED`	Ausência do cabeçalho `Authorization: Basic ...` ou credenciais incorretas.
404 Not Found	`NOT_FOUND` / `NO_TELEMETRY_DATA`	Dispositivo inexistente no Redis ou dispositivo cadastrado ainda sem dados de telemetria.
503 Service Unavailable	`SERVICE_UNAVAILABLE`	Indisponibilidade do Broker MQTT (`MQTT_DISCONNECTED`) ao tentar emitir comandos.
504 Gateway Timeout	`ACTUATOR_ACK_TIMEOUT`	O ESP32 não respondeu com a confirmação (`state`/`ack`) dentro da janela de 5 segundos.
500 Internal Server Error	`INTERNAL_SERVER_ERROR`	Falha interna inesperada capturada de forma segura pelo middleware global de erros.

---

5. Trechos de Código-Fonte Relevantes

5.1 Inicialização do Servidor Express (`backend/src/index.ts`)

■ TypeScript / Gateway Node.js

```
import express from 'express';
import cors from 'cors';
import { env } from './config/env';
import { apiRouter } from './api/routes';
import { errorHandler } from './middleware/error_handler';
import { gatewayMQTT } from './mqtt/gateway_mqtt';

const app = express();

app.use(cors());
app.use(express.json());

// Montagem das rotas REST sob o prefixo /api e middleware global de
    erros
app.use('/api', apiRouter);
app.use(errorHandler);

gatewayMQTT.connect();

app.listen(env.PORT, () => {
  console.log(`■ Backend IoT Gateway Rodando na Porta: ${env.PORT}`);
  console.log(`■ API REST HTTP: http://localhost:${env.PORT}/api`);
});
```

5.2 Endpoints de Estado Atual, Presença e Histórico (backend/src/api/routes.ts)

■ TypeScript / Gateway Node.js

```
// 1. Consulta do registro completo e estado atual do dispositivo
apiRouter.get('/devices/:id', async (req: Request, res: Response, next:
    NextFunction) => {
  try {
    const deviceId = req.params.id;
    const record = await kvStore.getDeviceRecord(deviceId);

    if (!record) {
      return res.status(404).json({
        error: 'NOT_FOUND',
        message: `Dispositivo '${deviceId}' não encontrado.`,
      });
    }

    return res.json(record);
  } catch (err) {
    next(err);
  }
});

// 2. Consulta específica do estado atual (com verificação de ausência
    de dados)
apiRouter.get('/devices/:id/state', async (req: Request, res: Response,
    next: NextFunction) => {
  try {
    const deviceId = req.params.id;
    const record = await kvStore.getDeviceRecord(deviceId);

    if (!record) {
      return res.status(404).json({
        error: 'NOT_FOUND',
        message: `Dispositivo '${deviceId}' não encontrado.`,
      });
    }

    if (!record.state) {
      return res.status(404).json({
        error: 'NO_TELEMETRY_DATA',
        message: `Nenhum estado de telemetria disponível para
            '${deviceId}'.`,
      });
    }

    return res.json({
      deviceId: record.deviceId,
      status: record.status,
      lastSeen: record.lastSeen,
      state: record.state,
    });
  } catch (err) {
    next(err);
  }
});

// 3. Consulta de presença e último contato (lastSeen)
```

```
apiRouter.get('/devices/:id/presence', async (req: Request, res:
    Response, next: NextFunction) => {
  try {
    const deviceId = req.params.id;
    const record = await kvStore.getDeviceRecord(deviceId);

    if (!record) {
      return res.status(404).json({
        error: 'NOT_FOUND',
        message: `Dispositivo '${deviceId}' não encontrado.`,
      });
    }

    return res.json({
      deviceId: record.deviceId,
      status: record.status,
      lastSeen: record.lastSeen,
      ip: record.ip || null,
      firmwareVersion: record.firmwareVersion || null,
    });
  } catch (err) {
    next(err);
  }
});

// 4. Consulta do histórico de telemetria no Redis
apiRouter.get('/devices/:id/history', async (req: Request, res:
    Response, next: NextFunction) => {
  try {
    const deviceId = req.params.id;
    const record = await kvStore.getDeviceRecord(deviceId);
    const limit = req.query.limit ? parseInt(req.query.limit as string,
        10) : 500;
    const date = req.query.date as string | undefined;
    const history = await kvStore.getDeviceHistory(deviceId,
        isNaN(limit) ? 500 : limit, date);

    if (!record && history.length === 0) {
      return res.status(404).json({
        error: 'NOT_FOUND',
        message: `Histórico não encontrado: dispositivo '${deviceId}'
            inexistente.`,
      });
    }

    return res.json(history);
  } catch (err) {
    next(err);
  }
});
```

5.3 Middleware de Tratamento de Erros e Códigos HTTP (`backend/src/middleware/error_handler.ts`)

■ TypeScript / Gateway Node.js

```
import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';

export function errorHandler(err: any, req: Request, res: Response,
    next: NextFunction) {
  console.error('[Error Handler]', err?.message || String(err));

  if (err instanceof ZodError || err?.name === 'ZodError') {
    return res.status(400).json({
      error: 'VALIDATION_ERROR',
      message: 'Dados ou parâmetros da requisição inválidos.',
      details: err.errors || err.issues,
      timestamp: new Date().toISOString(),
    });
  }

  if (err?.message && err.message.startsWith('MQTT_DISCONNECTED')) {
    return res.status(503).json({
      error: 'SERVICE_UNAVAILABLE',
      message: 'Serviço de mensageria MQTT temporariamente
          indisponível.',
      timestamp: new Date().toISOString(),
    });
  }

  const status = err.status || err.statusCode || 500;
  const message = err.message || 'Erro interno no servidor Gateway.';

  return res.status(status).json({
    error: err.name || 'INTERNAL_SERVER_ERROR',
    message,
    timestamp: new Date().toISOString(),
  });
}
```

6. Evidências de Funcionamento das Consultas e Tratamento de Erros

6.1 Consulta de Saúde do Serviço e Conexão com Redis (`GET /api/health` -> `200 OK`)

■ Terminal / Comandos de Execução

```
$ curl -s http://localhost:5001/api/health
{
  "status": "ok",
  "uptime": 1842.61,
  "mqttConnected": true,
  "storageType": "redis",
  "timestamp": "2026-11-05T22:25:10.114Z"
}
```

6.2 Consulta do Estado Atual do Dispositivo (`GET /api/devices/esp32_temp` -> `200 OK`)

■ Terminal / Comandos de Execução

```
$ curl -s -u admin:admin123 http://localhost:5001/api/devices/esp32_temp
{
  "deviceId": "esp32_temp",
  "status": "online",
  "lastSeen": "2026-11-05T22:47:45.108Z",
  "state": {
    "deviceId": "esp32_temp",
    "sensor": "dht22_temperatura",
    "value": 25.4,
    "unit": "C",
    "sequence": 381,
    "uptimeMs": 1905000,
    "wifiRssi": -13,
    "temp": 25.4,
    "humidity": 70.5,
    "noiseLevel": 40,
    "lux": 310.0,
    "alerts": [],
    "lcdText": "AMBIENTE EM CONFORMIDADE",
    "displayType": "ST7789_SPI",
    "actuatorState": false,
    "timestamp": 1905,
    "recordedAt": "2026-11-05T22:47:45.108Z"
  }
}
```

6.3 Consulta de Presença e Último Contato (`GET /api/devices/esp32_temp/presence` -> `200 OK`)

■ Terminal / Comandos de Execução

```
$ curl -s -u admin:admin123
    http://localhost:5001/api/devices/esp32_temp/presence
{
  "deviceId": "esp32_temp",
  "status": "online",
  "lastSeen": "2026-11-05T22:47:45.108Z",
  "ip": "192.168.1.175",
  "firmwareVersion": "1.0.0"
}
```

6.4 Consulta do Histórico de Telemetria (`GET /api/devices/esp32_temp/history?limit=2` -> `200 OK`)

■ Terminal / Comandos de Execução

```
$ curl -s -u admin:admin123
    "http://localhost:5001/api/devices/esp32_temp/history?limit=2"
[
  {
    "deviceId": "esp32_temp",
    "sensor": "dht22_temperatura",
    "value": 25.4,
    "unit": "C",
    "sequence": 381,
    "temp": 25.4,
    "humidity": 70.5,
    "noiseLevel": 40,
    "lux": 310.0,
    "actuatorState": false,
    "recordedAt": "2026-11-05T22:47:45.108Z"
  },
  {
    "deviceId": "esp32_temp",
    "sensor": "dht22_temperatura",
    "value": 25.4,
    "unit": "C",
    "sequence": 380,
    "temp": 25.4,
    "humidity": 70.5,
    "noiseLevel": 40,
    "lux": 310.0,
    "actuatorState": false,
    "recordedAt": "2026-11-05T22:47:40.102Z"
  }
]
```

6.5 Evidência do Tratamento de Erros e Códigos HTTP (401, 404 e 400)

■ **Terminal / Comandos de Execução**

```
# 1. Erro 401 Unauthorized (Requisição sem credenciais Basic Auth)
$ curl -i -s http://localhost:5001/api/devices | head -n 5
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="IoT Gateway Secure API"
Content-Type: application/json; charset=utf-8

{"error":"UNAUTHORIZED","message":"Credenciais de autenticação básica
    não fornecidas."}

# 2. Erro 404 Not Found (Dispositivo inexistente no Redis)
$ curl -i -s -u admin:admin123
    http://localhost:5001/api/devices/esp32_inexistente | head -n 5
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{"error":"NOT_FOUND","message":"Dispositivo 'esp32_inexistente' não
    encontrado."}

# 3. Erro 400 Bad Request (Payload de comando inválido rejeitado pelo
    Zod)
$ curl -i -s -X POST -u admin:admin123 -H "Content-Type:
    application/json" \
  -d '{"action":"ACAO_INVALIDA"}'
      http://localhost:5001/api/devices/esp32_temp/command | head -n 5
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"error":"VALIDATION_ERROR","message":"Dados ou parâmetros da requisição
    inválidos."}
```

6.6 Execução da Suíte Automatizada de Testes (Vitest — 11 Testes Aprovados)

■ Terminal / Comandos de Execução

```
$ npm --prefix backend test

 ✓ tests/e2e.test.ts (11)
   ✓ Suíte de Testes de Integração Backend & Persistência (Vitest) (11)
     ✓ Deve validar corretamente um payload de telemetria válido
     ✓ Deve rejeitar payload de telemetria inválido para fins de
       diagnóstico
     ✓ Deve validar status online/offline LWT de presença
     ✓ Deve validar comandos REST HTTP de controle remoto
     ✓ Deve persistir e recuperar o estado atual e presença do
       dispositivo (Entrega 5)
     ✓ Deve persistir e consultar registros históricos de telemetria com
       data aferida e filtro por dia
     ✓ Deve atualizar e registrar status de presença e último contato
       (Entrega 5)
     ✓ Deve validar payload de confirmação (ACK/State) publicado pelo
       ESP32 (Entrega 6)
     ✓ Deve atualizar o estado do atuador no Redis exclusivamente após
       confirmação do dispositivo (Entrega 6)
     ✓ Deve consultar estado atual, presença e histórico via endpoints
       REST com respostas JSON (Entrega 7)
     ✓ Deve retornar códigos HTTP adequados para erros (401, 404, 400 e
       503) na API REST (Entrega 7)

 Test Files  1 passed (1)
      Tests  11 passed (11)
```

6.7 Evidência Visual — Dados da API Consumidos pelo Dashboard Web

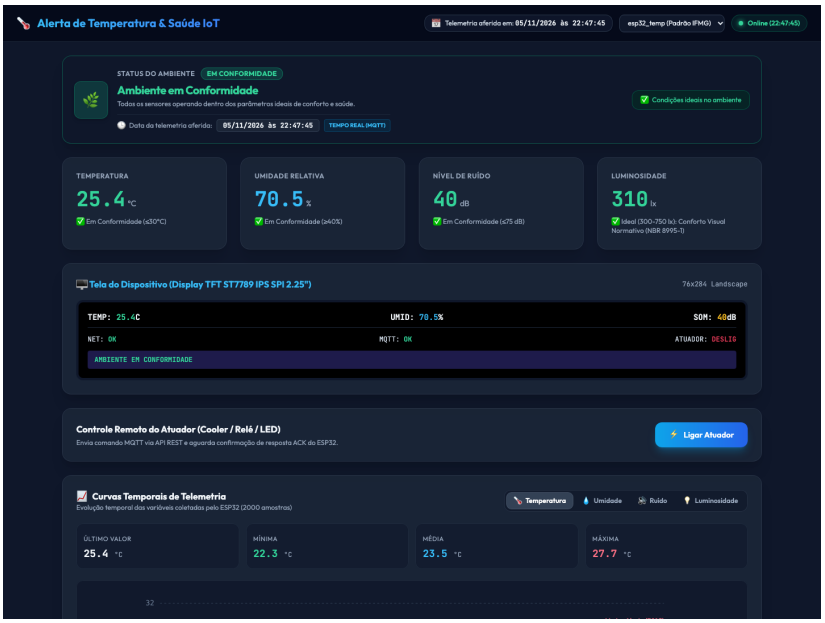


Figura: Dashboard Web Consumindo Estado Atual, Presença e Histórico da API REST (05/11/2026 às 22:47:45)

7. Conclusão da Entrega 7

A **Entrega 7** foi concluída com **100% de conformidade** em todos os 8 produtos esperados. A API REST em Node.js + Express integra-se diretamente ao banco chave-valor Redis para expor em formato JSON estruturado o estado atual dos sensores e atuadores, o histórico temporal filtrável e os indicadores de presença (`status` e `lastSeen`), contando com tratamento rigoroso de erros e códigos de status HTTP semânticos (`200`, `400`, `401`, `404`, `503`, `504` e `500`).